

A Suggestion for Reflection Access Control

Document #: P3451R0 [[Latest](#)] [[Status](#)]
Date: 2024-10-14
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Introduction](#)
- [2 Accessibility as Property](#)
- [3 What Does Equality Mean?](#)
- [4 Proposal](#)
- [5 References](#)

§ 1 Introduction

There is still ongoing discussion about what to do for reflection access in [[P2996R6](#)].

When it comes to access, there are two issues that we are stuck between:

1. We do want to give people access to *observe* the existence of private base classes or data members. Doing so is important for some use-cases, and observation of metadata is not, itself, problematic.
2. We do want to be able to limit access to splices, since directly reading from and, worse, writing to private members could break invariants and be problematic.

If we perform access checking at the point of retrieval, e.g. `accessible_members_of()`, then we can't check to see if private subobjects exist. You just can't implement some type traits, even if all those type traits want to do is reject the existence of private members.

If we perform access checking at the point of splice, that's inconsistent with how the language generally works and prohibits giving out access. For instance:

```
class C {  
private:  
    int i;  
  
public:  
    auto ref() -> int& { return i; }  
    static constexpr auto pmd() { return &C::i; }  
    static constexpr auto refl() { return ^^C::i; }  
};
```

```

void use(int);

void demo(C c) {
    use(c.i);      // error: access
    use(c.ref()); // ok: I got the member from elsewhere

    use(c.*C::i);  // error: access
    use(c.*C::pmd()); // ok: I got the pointer-to-member from elsewhere

    use(c.[:^C::i:]); // error: access
    use(c.[:C::refl():]); // should be ok: I got the reflection from elsewhere
}

```

This seems like we're kind of stuck.

§ 2 Accessibility as Property

But I think there's another option, inspired by Wyatt Childers but maybe something slightly different than what he was going for. A reflection of a base or a non-static data member already has several properties. Today, those properties are all just the actual properties of the thing - `^C::i` (if you can get it) is a non-static data member of type `int` in `C` that's `private` at offset `0`, etc.

We can add one more property to the reflection: can it be accessed?

```

constexpr bool is_accessible(info);

```

And an emergency escape hatch to force accessibility:

```

constexpr info force_accessibility(info);

```

This property works in the following way. Rather than `accessible_members_of()` or `get_public_members()` or whatever, we just have `members_of()` and `friends`. Those functions will imbue the returned reflections with the appropriate accessibility for non-public subobjects.

Splicing *will* check access, but in a way that can be worked around in the same way that pointers-to-members can be returned.

For example:

```

class C {
private:
    int i;

public:

```

```

// assuming we fix this to work in this context
static_assert(nonstatic_data_members_of(C).size() == 1);

// of course we have access to our own members
static_assert(is_accessible(nonstatic_data_members_of(C)[0]));

// C::i in this context is accessible, which is remembered
static constexpr auto get_refl() { return C::i; }
};

void demo(C c) {
    // we still see all of them
    static_assert(nonstatic_data_members_of(C).size() == 1);

    // I can get access to it this way, but not C::i
    constexpr auto r = nonstatic_data_members_of(C)[0];

    // it's still private
    static_assert(is_private(r));

    // and I can see other stuff about it
    static_assert(type_of(r) == int);
    static_assert(identifier_of(r) == "i");

    // but it's not accessible
    static_assert(not is_accessible(r));

    // ... which means I cannot splice it
    int i = c.[:r:]; // error

    // however, I can splice this one
    constexpr auto r2 = C::get_refl();
    static_assert(is_private(r2));
    static_assert(is_accessible(r2));
    int j = c.[:r2:]; // ok

    // or this one
    int k = c.[: force_accessibility(r) :];
}

```

In this way, splicing still checks access - but in a way that is more consistent with pointers to members. Access checking can still be explicitly bypassed, with a loud function that can be easily searched. And access can be propagated with `friendship` in a way that users expect. All while not preventing observation of private metadata.

So in general the rule could be that `c.[:r:]` works if:

- `is_accessible(r)` is `true`, or
- in the context of the splice the subobject designated by `r` is accessible

Note that accessibility isn't transitive. Imagine a situation like:

```
class A {
private:
    class B {
    private:
        int x;
    };
};
```

If `b` is a reflection of `A::B`, then what is the accessibility of the data members returned by `members_of(b)`? It should be the same as the answer for getting `members_of(^A)`. That is:

- If `members_of(b)` is invoked in a context in which `B::x` is accessible, then it is returned as accessible.
- Otherwise, it is inaccessible.

The accessibility of `b` is immaterial here. `members_of(b)` and `members_of(force_accessibility(b))` yield the same reflections, and `^B::x` could still be inaccessible in the latter.

§ 3 What Does Equality Mean?

The big question with this approach is: what does equality mean? That is:

```
class C {
    int i;

public:
    static constexpr auto get() { return ^C::i; }
};

constexpr auto outer = nonstatic_data_members_of(^C)[0];
constexpr auto inner = C::get();

static_assert(!is_accessible(outer));
static_assert(is_accessible(inner));

static_assert(outer == inner); // ???
```

Here, `outer` and `inner` both represent the non-static data member `C::i`. They only have one difference: `outer` is not accessible but `inner` is. Which is a fairly observable difference:

```
void observe(C c) {
    int x = c.[:outer:]; // error
    int y = c.[:inner:]; // ok
}
```

So, should they compare equal? The argument for no is that they are not substitutable, so they should not. And `x == y` should imply that `[:x:] == [:y:]` (or, in this case, that the appropriate `c.[:x:]` has the same meaning as `c.[:y:]`). The argument for yes is that this really isn't the most salient property in situations where you probably actually want equality, like `std::find()`-ing a non-static data member in a collection. And that pointers-to-members can already compare equal despite referring to different union alternatives.

I don't have much experience with using `==` with objects of type `info` yet. Most of my uses have been with comparing values (where the meaning of `==` is template-argument-equivalence) or types (where the question of substitutability has meant that a reflection of an alias does not compare equal to a reflection of its underlying type, which has been a frequent source of annoyance). So I could really go either way on this. Given that we're preserving alias for `==` (and aliases have far more distinct properties than accessibility), we should probably preserve accessibility for `==`.

§ 4 Proposal

1. Additionally add the concept of accessibility to reflections of base class subobjects and all members. A base class subobject or member reflection returned from the `members_of` family of functions satisfies `is_accessible` if the caller has access to that member at the point of call. Accessibility is part of equality.
2. Add the two new metafunctions:

```
constexpr bool is_accessible(info r);
constexpr info force_accessibility(info r);
constexpr info make_inaccessible(info r);
```

The first returns `false` if `r` is a reflection of a base class subobject or member that was not accessible from the point at which it was generated, otherwise `true` (this phrasing gets us `is_accessible(^std)`, which will make it more straightforward to word splice access checking).

The second and third return new reflections with all the same properties of `r` except that it is accessible and inaccessible, respectively. They have deliberately distinct names for ease of search. `force_accessibility` is valuable to bypass accessibility. `make_inaccessible` is going to be valuable to be able to do an `==` check without having to strip accessibility.

3. Change the rules for splicing as follows: `[:r:]` is valid if either `is_accessible(r)` is `true` or `r` represents a base class subobject or a member that is accessible from the point of splice.
4. Remove the `get_public_meow` family of metafunctions that we recently added, since this paper solves that problem better.

§ 5 References

[P2996R6] Wyatt Childers, Peter Dimov, Dan Katz, Barry Revzin, Andrew Sutton, Faisal Vali, and Daveed Vandevoorde. 2024-09-24. Reflection for C++26.

<https://wg21.link/p2996r6>